

AGRÉGATION — ÉPREUVE ORALE DE MODÉLISATION
OPTION INFORMATIQUE

Finalité et responsabilité dans un consensus réparti

Le protocole Casper

Texte rédigé d'après V. Buterin et V. Griffith, *Casper the Friendly Finality Gadget*,
Ethereum Foundation, arXiv:1710.09437v4, 2019.

Pr Agrégé El Hadiq Zouhair — cpgeacademy.org

Recommandations du jury. Il est rappelé que le jury n'exige pas une compréhension exhaustive du texte. Vous êtes laissé(e) libre d'organiser votre discussion comme vous l'entendez. Des suggestions de développement, largement indépendantes les unes des autres, vous sont proposées en fin de texte. Vous n'êtes pas tenu(e) de les suivre. Il vous est conseillé de mettre en lumière vos connaissances à partir du fil conducteur constitué par le texte. Le jury appréciera que la discussion soit accompagnée d'exemples traités sur ordinateur.

1 Le problème

Un *registre réparti* est une base de données répliquée sur un grand nombre de machines qui ne se font pas confiance, connectées par un réseau non fiable, et dont une fraction inconnue peut se comporter de manière arbitrairement malveillante (modèle *byzantin*). Le problème central est celui du **consensus** : faire converger tous les participants honnêtes vers une même suite de transactions.

Les systèmes de type *preuve de travail* (Bitcoin) résolvent ce problème de façon probabiliste : un bloc n'est jamais définitivement acquis, il devient seulement de plus en plus improbable qu'il soit révisé. À l'opposé, les algorithmes issus de la tradition BFT (PBFT, Tendermint) offrent une **finalité** déterministe : un bloc validé ne peut plus être révisé, sous l'hypothèse que moins d'un tiers des participants sont byzantins.

Transposer ces algorithmes au cadre de la *preuve d'enjeu*, où l'influence d'un participant est proportionnelle au capital qu'il immobilise, se heurte au problème dit du « *rien en jeu* » (*nothing at stake*) : voter simultanément pour deux branches concurrentes ne coûte rien, alors qu'en preuve de travail cela exigerait de dédoubler sa puissance de calcul. Le protocole **Casper** apporte une réponse à ce problème en rendant la triche non seulement détectable mais **imputable** : si deux blocs incompatibles sont finalisés, on sait *quels* participants ont fauté, et l'on peut confisquer leur capital.

Ce texte propose une modélisation de Casper. On y formalise l'objet combinatoire sur lequel le protocole opère (§2), on démontre ses deux propriétés fondamentales (§3), on examine les questions algorithmiques que soulève sa mise en œuvre (§4), puis on discute deux extensions et leurs limites (§5).

2 Le modèle

2.1 Arbre de blocs et points de contrôle

Le mécanisme qui *propose* les blocs (une chaîne à preuve de travail, dans la première version de Casper) est extérieur au protocole : Casper est une *surcouche*. On le modélise simplement par sa production.

Définition 1. Un **arbre de blocs** est un arbre enraciné fini $(\mathcal{B}, g)$; la racine g est le *bloc génésis*, et le *numéro* $\text{nb}(b)$ d'un bloc est sa profondeur dans $\mathcal{B}$.

En régime normal, le mécanisme de proposition produit une simple liste chaînée ; mais la latence du réseau ou une attaque délibérée engendrent inévitablement plusieurs fils d'un même bloc. La tâche de Casper est de choisir, dans cet arbre, une unique chaîne canonique.

Raisonnement sur l'arbre complet serait trop coûteux. On se restreint donc à un sous-arbre espacé :

Définition 2. Un bloc b est un **point de contrôle** si $b = g$ ou si $\text{nb}(b) \equiv 0 \pmod{100}$. Les points de contrôle, munis de la relation « plus proche ancêtre qui est un point de contrôle », forment un arbre $\mathcal{C}$ enraciné en $r = g$, l'**arbre des points de contrôle**. La **hauteur** $h(c)$ d'un point de contrôle est sa profondeur dans $\mathcal{C}$; ainsi $h(c) = \text{nb}(c)/100$.

On note $a \preceq b$ (« a est un ancêtre de b ») la relation d'ordre partiel induite par $\mathcal{C}$, et $a \prec b$ sa version stricte. Deux points de contrôle a et b sont **en conflit**, ce qu'on note $a \bowtie b$, lorsque ni $a \preceq b$ ni $b \preceq a$: ils appartiennent à deux branches distinctes.

Remarque 3. L'espacement de 100 blocs est un compromis : plus il est grand, plus le coût du protocole est amorti, mais plus la finalisation est lente. C'est un paramètre de modélisation, non une nécessité mathématique ; toute la théorie qui suit vaut pour un espacement quelconque.

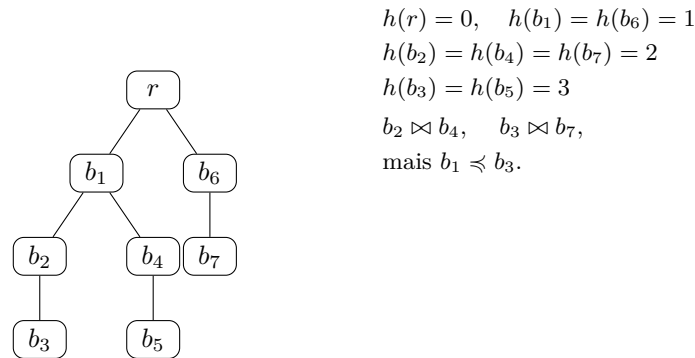


Figure 1 — Un arbre de points de contrôle. Chaque arête représente 99 blocs intermédiaires.

2.2 Validateurs, dépôts et votes

Définition 4. Un **jeu de validateurs** est un ensemble fini $\mathcal{V}$ muni d'une application $d : \mathcal{V} \rightarrow \mathbb{N}^*$, le *dépôt*. On pose, pour $E \subseteq \mathcal{V}$, $w(E) = \sum_{\nu \in E} d(\nu)$ et $T = w(\mathcal{V})$. Une partie E est une **supermajorité** lorsque $w(E) \geq \frac{2}{3}T$.

C'est ici le premier choix de modélisation notable : la sécurité d'un système à preuve d'enjeu ne dépend pas du *nombre* de validateurs mais du *capital* immobilisé. On raisonne donc systématiquement avec la mesure w , et jamais avec le cardinal. Toutes les fractions $(\frac{2}{3}, \frac{1}{3})$ s'entendent au sens de w .

Définition 5. Un **vote** est un quadruplet $\langle \nu, s, t, h(s), h(t) \rangle$ signé par ν , où $\nu \in \mathcal{V}$, et $s, t \in \mathcal{C}$. Il est **valide** lorsque $s \prec t$. On dit que s est la *source* et t la *cible*. Pour $(s, t) \in \mathcal{C}^2$, on note $E_{s \rightarrow t}$ l'ensemble des validateurs ayant publié un vote valide de source s et de cible t .

Les hauteurs $h(s)$ et $h(t)$ sont redondantes avec s et t : elles figurent dans le message pour que les conditions de la §3 soient vérifiables *sans connaître l'arbre*, donc localement et à moindre coût. C'est un point de modélisation important : les *conditions de sanction* ne portent que sur les hauteurs, jamais sur la topologie de l'arbre ; elles sont donc indépendantes de l'état de la chaîne.

Définition 6. Un couple (s, t) est un **lien de supermajorité**, noté $s \rightarrow t$, lorsque $E_{s \rightarrow t}$ est une supermajorité. On note $\mathcal{L} \subseteq \mathcal{C}^2$ l'ensemble des liens de supermajorité.

L'ensemble $\mathcal{J}$ des points de contrôle **justifiés** est la plus petite partie de $\mathcal{C}$ telle que

$$r \in \mathcal{J} \quad \text{et} \quad (c \in \mathcal{J} \text{ et } c \rightarrow c') \implies c' \in \mathcal{J}.$$

Un point de contrôle c est **finalisé** lorsque $c = r$, ou lorsque $c \in \mathcal{J}$ et qu'il existe c' fils direct de c dans $\mathcal{C}$ tel que $c \rightarrow c'$. On note $\mathcal{F}$ l'ensemble des points de contrôle finalisés.

Remarque 7. La définition de $\mathcal{J}$ est celle d'un *plus petit point fixe* : l'opérateur $\Phi(X) = \{r\} \cup \{c' : \exists c \in X, c \rightarrow c'\}$ est monotone sur le treillis complet $(\mathcal{P}(\mathcal{C}), \subseteq)$, donc admet par le théorème de Knaster–Tarski un plus petit point fixe, qui vaut $\bigcup_{k \geq 0} \Phi^k(\emptyset)$ puisque $\mathcal{C}$ est fini. On reconnaît par ailleurs, en lisant $\mathcal{L}$ comme l'ensemble des arcs d'un graphe orienté sur $\mathcal{C}$, que $\mathcal{J}$ est exactement l'ensemble des sommets *accessibles depuis* r . Les deux lectures — point fixe et accessibilité — conduisent à deux implantations différentes (§4).

Exemple 8. Sur la figure 1, supposons $\mathcal{V} = \{\alpha, \beta, \gamma, \delta\}$ de dépôts respectifs 30, 30, 20, 20, donc $T = 100$ et le seuil de supermajorité $\lceil 2T/3 \rceil = 67$. Si α, β, γ (de poids 80) votent tous trois $r \rightarrow b_1$, $b_1 \rightarrow b_2$ et $b_2 \rightarrow b_3$, alors $\mathcal{L} = \{(r, b_1), (b_1, b_2), (b_2, b_3)\}$, $\mathcal{J} = \{r, b_1, b_2, b_3\}$ et $\mathcal{F} = \{r, b_1, b_2\}$: le point b_3 est justifié mais pas encore finalisé.

Notons qu'un lien peut « sauter » des points de contrôle : rien n'impose $h(t) = h(s) + 1$ dans la définition 6. C'est précisément ce qui rend le protocole robuste à la perte de messages, et c'est aussi ce qui rend les démonstrations qui suivent non triviales.

3 Sûreté responsable et vivacité

3.1 Les deux conditions de sanction

Définition 9. Un validateur ν **faute** s'il publie deux votes distincts $\langle \nu, s_1, t_1 \rangle$ et $\langle \nu, s_2, t_2 \rangle$ vérifiant l'une des deux conditions :

- (I) $h(t_1) = h(t_2)$: deux votes distincts pour une même hauteur de cible ;
- (II) $h(s_1) < h(s_2) < h(t_2) < h(t_1)$: un vote strictement emboîté dans un autre.

On note Faut l'ensemble des validateurs fautifs. La preuve d'une faute étant un couple de messages signés, elle peut être publiée dans la chaîne elle-même ; le protocole confisque alors l'intégralité du dépôt du fautif.

La condition (I) interdit de soutenir deux branches concurrentes à un même niveau. La condition (II) est plus subtile : elle interdit de « recouvrir » un vote par un autre, ce qui reviendrait à revenir sur une décision déjà prise. Ces deux conditions sont *locales* (elles ne concernent qu'un validateur à la fois) et *sans état* (elles ne portent que sur les quatre hauteurs des deux messages).

3.2 Le lemme d'intersection

Tout repose sur l'observation élémentaire suivante.

Lemme 10 (Intersection). *Si E_1 et E_2 sont deux supermajorités, alors $w(E_1 \cap E_2) \geq \frac{T}{3}$.*

Démonstration. La mesure w étant additive, $w(E_1 \cup E_2) = w(E_1) + w(E_2) - w(E_1 \cap E_2)$. Comme $E_1 \cup E_2 \subseteq \mathcal{V}$ et que w est croissante, $w(E_1 \cup E_2) \leq T$, d'où $w(E_1 \cap E_2) \geq \frac{2}{3}T + \frac{2}{3}T - T = \frac{T}{3}$. $\square$

Ce lemme est le seul endroit où le seuil $\frac{2}{3}$ intervient ; il explique pourquoi c'est ce seuil, et pas un autre, qui est retenu. Le tiers ainsi dégagé est la « caution » : c'est lui qui sera détruit en cas d'attaque.

Proposition 11. *Supposons $w(\text{Faut}) < T/3$. Alors :*

- (i) si $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ sont deux liens distincts, alors $h(t_1) \neq h(t_2)$;
- (ii) si $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ sont deux liens distincts, l'inégalité $h(s_1) < h(s_2) < h(t_2) < h(t_1)$ est impossible ;
- (iii) pour toute hauteur n , il existe au plus un lien de supermajorité de hauteur de cible n ;
- (iv) pour toute hauteur n , il existe au plus un point de contrôle justifié de hauteur n .

Démonstration. (i) Si $h(t_1) = h(t_2)$, le lemme 10 appliqué à $E_{s_1 \rightarrow t_1}$ et $E_{s_2 \rightarrow t_2}$ donne $w(E_{s_1 \rightarrow t_1} \cap E_{s_2 \rightarrow t_2}) \geq T/3$; or tout validateur de cette intersection a publié les deux votes distincts en cause, et faute donc par (I). D'où $w(\text{Faut}) \geq T/3$, contradiction. (ii) est identique avec la condition (II). (iii) est une reformulation de (i). (iv) : pour $n = 0$ seul r convient ; pour $n \geq 1$, deux justifiés distincts de hauteur n seraient les cibles de deux liens distincts de même hauteur de cible, ce qu'exclut (iii). $\square$

3.3 Le théorème de sûreté responsable

Théorème 12 (Sûreté responsable). *Si deux points de contrôle en conflit sont tous deux finalisés, alors $w(\text{Faut}) \geq T/3$.*

Autrement dit : une violation de la sûreté est impossible tant que moins d'un tiers du capital triche, et, si elle survient, au moins un tiers du capital total est détruit — et l'on sait à qui il appartenait.

Démonstration. Raisonnons par l'absurde en supposant $w(\text{Faut}) < T/3$, de sorte que la proposition 11 s'applique. Soient a_m et b_n deux points de contrôle finalisés avec $a_m \bowtie b_n$, de fils directs a_{m+1} et b_{n+1} tels que $a_m \rightarrow a_{m+1}$ et $b_n \rightarrow b_{n+1}$. Ces fils sont justifiés, puisque a_m et b_n le sont.

Cas $h(a_m) = h(b_n)$. Alors a_m et b_n sont deux justifiés distincts de même hauteur, ce qui contredit (iv).

Cas $h(a_m) < h(b_n)$ (quitte à échanger les rôles). Comme $b_n \in \mathcal{J}$, la définition 6 fournit une chaîne

$$r = \beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_k = b_n$$

de liens de supermajorité dont tous les termes sont justifiés ; les votes étant valides, on a $\beta_i \prec \beta_{i+1}$, donc $(h(\beta_i))_i$ est strictement croissante et chaque β_i est un ancêtre de b_n .

Étape 1. Aucune hauteur $h(\beta_i)$ ne vaut $h(a_m)$ ni $h(a_{m+1})$. En effet, si $h(\beta_i) = h(a_m)$, alors (iv) donne $\beta_i = a_m$, donc $a_m \preceq b_n$, ce qui contredit $a_m \bowtie b_n$; et si $h(\beta_i) = h(a_{m+1})$, alors $\beta_i = a_{m+1}$ et $a_m \preceq a_{m+1} \preceq b_n$, même contradiction.

Étape 2. On a $h(\beta_0) = 0 \leq h(a_m)$ et $h(\beta_k) = h(b_n) > h(a_m)$, donc, l'étape 1 excluant les égalités, $h(b_n) > h(a_{m+1})$. L'ensemble $\{i : h(\beta_i) > h(a_{m+1})\}$ est donc non vide ; soit j son minimum. Alors $j \geq 1$ et, par minimalité et par l'étape 1, $h(\beta_{j-1}) < h(a_m)$ (les valeurs $h(a_m)$ et $h(a_{m+1}) = h(a_m) + 1$ étant exclues).

Étape 3. On dispose ainsi des deux liens de supermajorité $\beta_{j-1} \rightarrow \beta_j$ et $a_m \rightarrow a_{m+1}$, avec

$$h(\beta_{j-1}) < h(a_m) < h(a_{m+1}) < h(\beta_j),$$

configuration exclue par (ii). Contradiction. $\square$

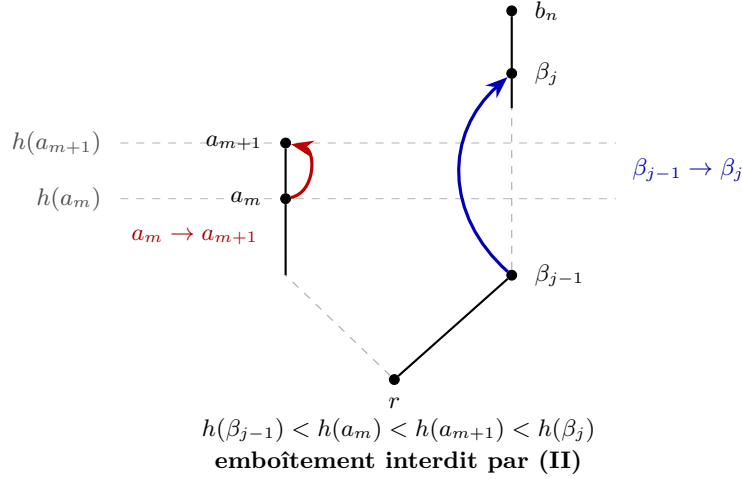


Figure 2 — Le cœur de la preuve du théorème 12 : le lien bleu enjambe strictement le lien rouge.

3.4 Vivacité plausible et règle de choix de fourche

La sûreté seule est vide de sens : un protocole qui ne finalise jamais rien est trivialement sûr. Il faut lui adjoindre une garantie de progrès.

Théorème 13 (Vivacité plausible). *Supposons qu'au moins $\frac{2}{3}$ des validateurs suivent le protocole, et que ceux-ci ne votent qu'avec une source justifiée. Alors, quel que soit l'historique des messages déjà émis, il est toujours possible d'ajouter des liens de supermajorité finalisant un nouveau point de contrôle, sans qu'aucun validateur ne faute.*

Démonstration. Soient a le justifié de plus grande hauteur, et b la cible de plus grande hauteur ayant déjà fait l'objet d'un vote. Soit a' un descendant de a avec $h(a') = h(b) + 1$ (il en existe dès que le mécanisme de proposition continue à produire des blocs). Le vote $a \rightarrow a'$ ne viole pas (I), car aucune cible de hauteur $h(b) + 1$ n'a encore été votée ; il ne viole pas (II) « par l'extérieur », car toute cible antérieure t vérifie $h(t) \leq h(b) < h(a')$, ni « par l'intérieur », car toute source antérieure s est justifiée donc vérifie $h(s) \leq h(a)$. On justifie ainsi a' ; on finalise ensuite a' en ajoutant le lien $a' \rightarrow a''$ vers un fils direct de a' , qui pour les mêmes raisons ne viole aucune condition. $\square$

Remarque 14. L'hypothèse « les validateurs honnêtes ne votent qu'avec une source justifiée » est implicite dans le texte original ; sans elle, un vote antérieur de source haute et de cible basse pourrait se trouver emboîté dans le nouveau vote. Sa portée exacte, et la façon dont le protocole l'impose, méritent discussion.

Le théorème 13 n'est pas seulement rassurant : il dicte la règle de choix de branche, qui est ainsi *correcte par construction*.

RÈGLE DE CHOIX DE FOURCHE. *Suivre la branche contenant le point de contrôle justifié de plus grande hauteur.*

D'après (iv), ce point de contrôle est unique ; d'après le théorème 13, c'est au-dessus de lui, et de lui seul, qu'il est toujours possible de progresser. Un participant qui suivrait la règle usuelle « la plus longue chaîne » pourrait au contraire se retrouver sur une branche où plus rien ne peut être finalisé sans que quelqu'un sacrifie son dépôt.

4 Aspects algorithmiques

4.1 Calcul des justifiés et des finalisés

L'arbre $\mathcal{C}$ est représenté par le tableau des pères, numéroté de sorte que le père d'un nœud porte toujours un numéro strictement plus petit — ce qui permet de calculer toutes les hauteurs en un seul balayage, en $\Theta(n)$. Les relations $a \preccurlyeq b$ et $a \bowtie b$ se testent en remontant les liens de parenté, en $O(h(b))$; le plus proche ancêtre commun s'obtient en $O(h(a) + h(b))$ en égalisant d'abord les hauteurs.

Le calcul de $\mathcal{L}$ à partir d'une liste de m votes est une simple agrégation par clé :

```

1 Algorithme LIENS( $votes, d, T$ )
2    $P \leftarrow$  dictionnaire vide ;  $U \leftarrow$  ensemble vide
3   pour chaque vote  $(\nu, s, t)$  de  $votes$  faire
4     si  $(\nu, s, t) \notin U$  alors # un validateur ne compte qu'une fois
5        $U \leftarrow U \cup \{(\nu, s, t)\}$  ;  $P[(s, t)] \leftarrow P[(s, t)] + d(\nu)$ 
6   renvoyer  $\{(s, t) : 3P[(s, t)] \geq 2T\}$ 

```

soit $O(m)$ en moyenne. Le test $3w \geq 2T$ évite tout flottant : les dépôts réels, exprimés en *wei* (10^{18} par ether), dépassent largement la précision des flottants double précision.

Le calcul de $\mathcal{J}$ suit la remarque faite après la définition 6. Deux implantations naturelles s'opposent :

- *itération du point fixe* : calculer $\Phi(\emptyset), \Phi^2(\emptyset), \dots$ jusqu'à stabilisation. Simple, directement calqué sur la définition, mais coûteux : chaque itération relit tout $\mathcal{L}$, d'où $O(|\mathcal{C}| \cdot |\mathcal{L}|)$ au pire ;
- *parcours de graphe* : construire les listes d'adjacence de $(\mathcal{C}, \mathcal{L})$ et effectuer un parcours depuis r , en $O(|\mathcal{C}| + |\mathcal{L}|)$.

Le passage de l'une à l'autre est un exemple typique du gain obtenu en reconnaissant, derrière une définition inductive, une structure de graphe. $\mathcal{F}$ s'en déduit en $O(|\mathcal{L}|)$: il suffit de parcourir les liens (c, c') et de retenir c lorsque $c \in \mathcal{J}$ et que c' est le fils direct de c .

4.2 Détection des fautes

La condition (I) se teste en $O(m)$ en moyenne : pour chaque validateur, on indexe ses votes par hauteur de cible et l'on signale toute collision avec une cible différente.

La condition (II) est plus intéressante. Testée naïvement, elle demande $\Theta(m_\nu^2)$ comparaisons pour un validateur ayant émis m_ν votes. Or elle admet la caractérisation suivante, pour un validateur donné dont on note L la liste des votes :

$$\nu \text{ faute par (II)} \iff \exists (s_2, t_2) \in L, \quad \max \{ h(t_1) : (s_1, t_1) \in L, h(s_1) < h(s_2) \} > h(t_2).$$

L'inégalité manquante $h(s_2) < h(t_2)$ est en effet automatique pour un vote *valide*. Il suffit alors de trier L par hauteur de source croissante et de balayer une fois en maintenant le maximum courant des $h(t)$, en traitant par blocs les votes de même hauteur de source pour respecter la stricte inégalité : on obtient un algorithme en $O(m \log m)$.

```

1 Algorithme FAUTE-II( $L, h$ ) #  $L$  : votes d'un meme validateur
2   trier  $L$  par  $h(s)$  croissant ;  $M \leftarrow -\infty$  ;  $u \leftarrow$  indéfini
3   pour chaque bloc  $B$  de votes de meme hauteur de source, dans l'ordre, faire
4     pour chaque  $(s_2, t_2)$  de  $B$  faire
5       si  $M > h(t_2)$  alors renvoyer le temoin  $(u, (s_2, t_2))$ 
6     pour chaque  $(s_1, t_1)$  de  $B$  faire
7       si  $h(t_1) > M$  alors  $M \leftarrow h(t_1)$  ;  $u \leftarrow (s_1, t_1)$ 
8   renvoyer "aucune faute"

```

4.3 Le coût d’une attaque

Le lemme 10 donne une borne inférieure sur le capital *détruit* ; une autre question est celle du capital à *mobiliser*. Un attaquant doit réunir une coalition $S \subseteq \mathcal{V}$ avec $w(S) \geq C$, où $C = \lceil 2T/3 \rceil$, et cherche naturellement à minimiser $w(S)$. On est ramené au problème d’optimisation

$$\mu = \min\{w(S) : S \subseteq \mathcal{V}, w(S) \geq C\},$$

dont le problème de décision associé contient SOMME-DE-SOUS-ENSEMBLE, donc est NP-difficile. Une programmation dynamique sur les sommes écrêtées à C le résout en $O(|\mathcal{V}| \cdot C)$ temps et $O(C)$ espace : c’est un algorithme *pseudo-polynomial*, inutilisable tel quel si C est exprimé en wei, mais parfaitement praticable après changement d’unité. On notera au passage que $\mu > C$ en général : la granularité des dépôts interdit d’atteindre exactement le seuil.

5 Deux extensions et leurs limites

5.1 Jeu de validateurs dynamique

Le modèle de la §2 suppose $\mathcal{V}$ fixe, ce qui est irréaliste. Casper indexe les entrées et sorties par la **dynastie** d’un bloc b , définie comme le nombre de points de contrôle finalisés dans la chaîne de r au père de b . Un validateur dont le message de dépôt est inclus dans un bloc de dynastie δ entre en fonction à la dynastie $DS(\nu) = \delta + 2$; symétriquement, un message de retrait fixe $DE(\nu)$. On définit alors deux jeux :

$$\mathcal{V}_f(\delta) = \{\nu : DS(\nu) \leq \delta < DE(\nu)\}, \quad \mathcal{V}_r(\delta) = \{\nu : DS(\nu) < \delta \leq DE(\nu)\},$$

de sorte que $\mathcal{V}_f(\delta) = \mathcal{V}_r(\delta + 1)$. Un lien $s \rightarrow t$ avec t de dynastie δ n’est reconnu que si *les deux* jeux $\mathcal{V}_f(\delta)$ et $\mathcal{V}_r(\delta)$ fournissent chacun une supermajorité.

Ce « recouvrement » n’est pas une précaution gratuite. Sans lui, deux petits-fils d’un même point finalisé peuvent se voir attribuer des dynasties différentes selon que la preuve de finalisation a été incluse ou non dans leur branche ; les deux jeux de validateurs correspondants peuvent alors être *disjoints*, et deux points de contrôle en conflit se trouvent finalisés sans qu’aucun validateur n’ait fauté. L’hypothèse du lemme d’intersection — les deux supermajorités se rapportent au même $\mathcal{V}$ — est violée, et le théorème 12 tombe. Le recouvrement force les deux jeux à se chevaucher, et restaure l’hypothèse.

5.2 Révisions lointaines

Une coalition ayant détenu plus de $\frac{2}{3}$ des dépôts dans un passé lointain, puis les ayant retirés, peut fabriquer après coup une chaîne concurrente : elle ne risque plus rien, son capital n’étant plus immobilisé. Casper s’en prémunit par deux ingrédients : une règle « ne jamais réviser un point finalisé », et un *délai de retrait* ω pendant lequel le dépôt reste confiscable. Sous les hypothèses d’une latence bornée par δ , d’horloges synchronisées et de blocs horodatés, on montre qu’il suffit que $\omega > 4\delta$ pour que tout client rejette la chaîne fabriquée. C’est le premier endroit du protocole où une hypothèse de *synchronie* apparaît : la sûreté du théorème 12, elle, n’en demandait aucune.

5.3 Fuite d’inactivité

Si plus d’un tiers des validateurs cessent de voter — panne, partition, censure — aucune supermajorité ne peut plus se former et la finalisation s’arrête indéfiniment : le protocole est bloqué sans que personne n’ait triché. Casper introduit alors une **fuite d’inactivité** : à chaque époque, le dépôt D d’un validateur silencieux est diminué de pD , de sorte qu’après k époques il vaut $D_k = D(1 - p)^k$. La série $\sum_k pD_k$ converge vers D : la fuite confisque asymptotiquement tout le dépôt d’un absent définitif, sans jamais le confisquer en temps fini. Le total T décroît, le seuil

$\lceil 2T/3 \rceil$ avec lui, et les validateurs restés actifs redeviennent une supermajorité en un nombre d'époques logarithmique en le rapport des dépôts.

Le prix à payer est élevé, et c'est peut-être le point le plus instructif du protocole. Lors d'une partition du réseau en $\mathcal{V}_A$ et $\mathcal{V}_B$, chaque camp voit l'autre comme inactif et lui applique la fuite. Au bout d'un certain temps, $\mathcal{V}_A$ est une supermajorité *selon les comptes de la branche A* et $\mathcal{V}_B$ une supermajorité *selon ceux de la branche B* : deux points de contrôle en conflit sont finalisés, *sans qu'aucun validateur n'ait enfreint la moindre condition*, donc sans qu'aucun dépôt ne soit détruit. Une fois encore, c'est l'hypothèse du lemme 10 — un même T pour les deux supermajorités — qui est en défaut.

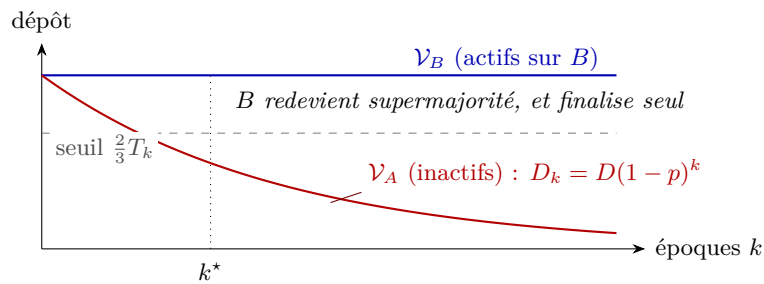


Figure 3 — Fuite d'inactivité : la vivacité est restaurée sur chaque branche, au prix de la sûreté entre branches.

Casper échange donc une sûreté *inconditionnelle* contre une sûreté *sous hypothèse de synchronie faible*, assortie d'une garantie de vivacité — arbitrage classique et inévitable, qu'éclairent le théorème CAP et le théorème d'impossibilité FLP. Le protocole prescrit alors une règle purement locale (« retenir le premier point finalisé que l'on a vu ») et renvoie l'arbitrage entre les deux branches hors du protocole, à une décision humaine.

Suggestions pour le développement

Attention. Il s'agit d'un menu à la carte : vous pouvez choisir d'étudier certains points, pas tous, pas nécessairement dans l'ordre, et de façon plus ou moins approfondie. Vous pouvez aussi vous poser d'autres questions que celles indiquées ci-dessous. Il est très vivement souhaité que vos investigations comportent une partie traitée sur ordinateur et, si possible, des représentations graphiques de vos résultats.

1. Implémenter l'arbre des points de contrôle et les primitives associées (hauteurs, ancêtre, conflit, plus proche ancêtre commun). Comparer expérimentalement le calcul des hauteurs nœud par nœud et le balayage unique, sur des arbres aléatoires de formes variées (chemin, arbre binaire complet, arbre de Galton–Watson).
2. Comparer les deux implantations du calcul de $\mathcal{J}$ évoquées en §4 (itération du point fixe, parcours de graphe) : mesurer les temps sur des instances aléatoires et confronter aux complexités annoncées. Que devient la comparaison si l'on veut recalculer $\mathcal{J}$ après l'ajout d'un seul lien ?
3. Écrire un vérificateur de fautes et le tester : engendrer aléatoirement des jeux de votes valides, comparer la détection quadratique de la condition (II) à la version en $O(m \log m)$, et vérifier que les témoins produits satisfont bien la définition 9.
4. Illustrer le théorème 12 par une recherche exhaustive : sur de petits arbres et de petits jeux de validateurs, énumérer les ensembles de votes finalisant deux points en conflit, et mesurer la distribution du capital effectivement confisqué. Retrouve-t-on la borne $T/3$? Est-elle atteinte ?

5. Simuler un réseau de validateurs (honnêtes, silencieux, byzantins) avec une latence aléatoire, et mesurer le délai de finalisation en fonction de la proportion de fautifs et de la latence. Que se passe-t-il au voisinage du seuil $\frac{1}{3}$?
6. Étudier la fuite d'inactivité : simuler une partition du réseau, tracer l'évolution des dépôts sur chaque branche et déterminer l'instant où chacune finalise. Comparer différentes formules de fuite (linéaire, géométrique, quadratique en la durée de non-finalisation).
7. Traiter le problème du coût minimal d'une attaque : programmation dynamique pseudo-polynomiale, comparaison avec une heuristique gloutonne et avec une recherche exhaustive. Étudier l'écart $\mu - C$ lorsque les dépôts suivent une loi de puissance, comme c'est le cas en pratique.
8. Formaliser la définition 6 comme plus petit point fixe et discuter l'apport du théorème de Knaster–Tarski ; envisager une spécification du protocole dans un formalisme vérifiable (automate, logique temporelle) et vérifier la sûreté par exploration exhaustive sur de petites instances.
9. Discuter la nécessité du seuil : montrer qu'aucun protocole de consensus ne peut tolérer un tiers ou plus de participants byzantins dans un réseau asynchrone, et comparer l'énoncé obtenu au théorème 12.
10. Comparer Casper à PBFT et à Tendermint : hypothèses de synchronie, nombre de messages échangés par décision, nature de la finalité obtenue, et présence ou non d'une notion d'imputabilité.

Références

- [1] V. BUTERIN, V. GRIFFITH, *Casper the Friendly Finality Gadget*, arXiv:1710.09437v4, Ethereum Foundation, 2019.
- [2] M. CASTRO, B. LISKOV, *Practical Byzantine Fault Tolerance*, OSDI, 1999.
- [3] J. KWON, *Tendermint: Consensus without Mining*, 2014.
- [4] M. FISCHER, N. LYNCH, M. PATERSON, *Impossibility of Distributed Consensus with One Faulty Process*, J. ACM, 1985.
- [5] S. NAKAMOTO, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
- [6] M. GAREY, D. JOHNSON, *Computers and Intractability*, Freeman, 1979 (pour SOMME-DE-SOUS-ENSEMBLE).